Foundations in Data Engineering

Performance and Efficiency – Part 2

Jana Giceva



Toy Example – Computing the Sum

The Toy Problem

- As example, we take the TPC-H dataset, relation `lineitem` and `sum up quantity` (5th column).

  1|155190|7706|1|17|21168.23|0.04|0.02|N|0|1996-03-13|…
  1|67310|7311|2|36|45983.16|0.09|0.06|N|0|1996-04-12|…
  1|63700|3701|3|8|13309.60|0.10|0.02|N|0|1996-01-29|…
  1|2132|4633|4|28|28955.64|0.09|0.06|N|0|1996-04-21|…
  …

- 725MB * SF (total data set roughly 1GB * SF)
- 6 million * SF lines
- We use text input for now
- Real benchmark is much more complex, of course
- But, that simple query can be analyzed easily and is surprisingly expensive

Learning Objectives

- Explain why both data- and thread-level parallelism are needed to approach the hardware limit, and be able to map each on to the available hardware mechanisms.
- Describe the SIMD execution models, compare the three ways to write vectorized code, and use SIMD operations to implement core data processing tasks.
- Apply the four design decisions of task-level parallelism to a concrete program: work partitioning, task-granularity, scheduling and correctness.
- Measure parallel performance and use Amdahl's law and be able to identify the common pitfalls.

Performance Limits

- What are the limits for that query (SF1, 725 MB)? Let's assume this hardware.

	Bandwidth	Query time
1Gbps ethernet	1Gbit/s	6s
HDD	200 MB/s	3.6s
SATA SSD	500 MB/s	1.6s
NVMe SSD	2 GB/s	0.36s
DRAM	32 GB/s	0.02s

- As you recall, we've just achieved 0.6s with C++ -O3, down from 4.4s with python.
  - **Q: Why was python so much slower than C++?**
  - **Q: Why was the first run always slower than the second?**

- Unfortunately, even with C++ -O3 we are still far from what our machine could do.

Saturating the memory bandwidth?

- **A single thread typically:**
  - Issues one or a few memory loads at a time.
  - Waits until the data arrives
  - And then continues
  - → only a handful of memory requests are in flight → memory channels stay mostly idle.

- **Let's do the math:**
  - Latency per DRAM bandwidth – 60ns
  - Size per access – 64 B (one cacheline)
  - Outstanding requests / thread (considering ILP) – ~10
  - Throughput of reading from memory

- **To saturate bandwidth, you'd need *hundreds* of outstanding loads so the memory controller can stream data continuously, and one thread cannot do that.**

Classes of Parallelism and Parallel Architectures

We distinguish two kinds of parallelism in applications:

- **Data-Level parallelism (DLP)**
  - many data items are processed at the same time
- **Task-Level parallelism (TLP)**
  - different tasks operate independently and in parallel

Classes of Parallelism and Parallel Architectures

Computer hardware can exploit them in four major ways:

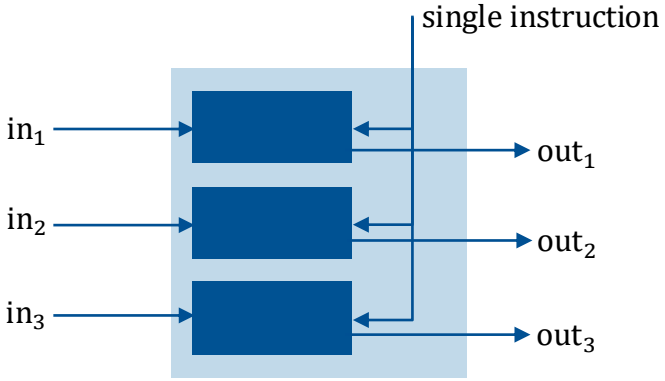
- **Instruction-level parallelism** (ILP)
  - exploits DLP using pipelining and speculative execution
- **Vector architectures** (SIMD, GPUs)
  - exploit DLP by applying the same instructions to a collection of data
- **Thread-level parallelism** (threads)
  - exploit DLP and/or TLP using hardware threads that work in parallel
- **Request-level parallelism** (distributed computing)
  - exploits DLP and TLP using mainly de-coupled tasks specified by the programmer (usually at large scale, think data-center)

Data-level parallelism
SIMD

Vector Processors (and SIMD extensions)

Most modern processors include a **SIMD** unit.

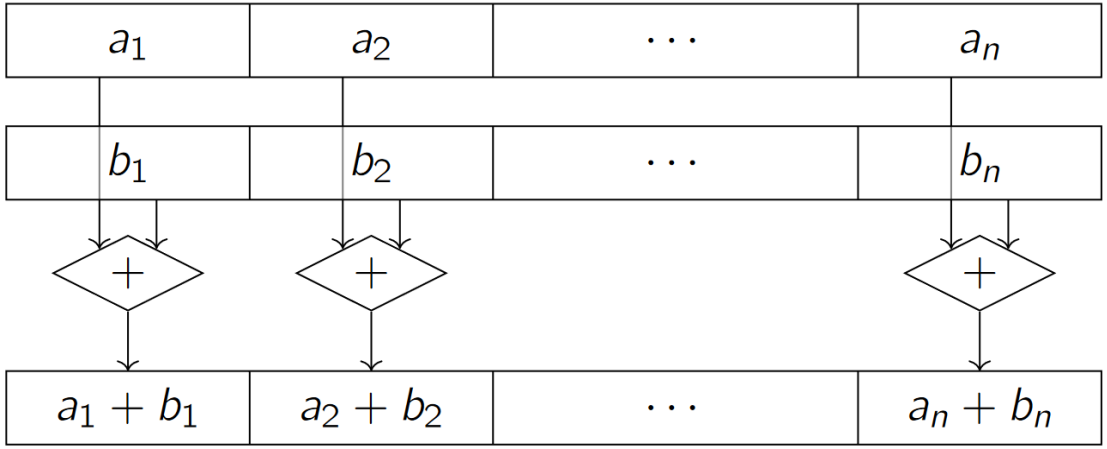
- Single Instruction stream, Multiple Data streams



- Execute the same assembly instructions on a set of values.
- Also called **vector unit; vector processors** are entire systems built on that idea.

SIMD Programming Model

The processing model is typically based on **SIMD registers** or **vectors.**



Typical values (*e.g.*, on Intel Skylake):

- 32×512 bit-wide registers (`zmm0` through `zmm31`)
- Usable as 64×8-bit integers, 32×16-bit integers, 16×32-bit integers, 8×64-bit integers, 16×32-bit floats, or 8×64-bit floats

SIMD Programming Model

- SIMD instructions make **independence** explicit
  - No data hazards within a vector instruction
  - Check for data hazards only between vectors.
  - **Data parallelism**

- Data **may** need to be **aligned** in memory to the **width** of the SIMD unit to prevent the compiler generating scalar instructions for otherwise vectorizable code.

- Parallel execution promises n-fold performance advantage
  - Not quite achievable in practice, however.

- Vector code sometimes uses more instructions on trivial things:
  - Converting and moving data to the right position in the register
  - Emulating branches with conditional moves

Coding for SIMD

How can I make use of SIMD instructions as a programmer?

- **Auto-vectorization**
  - Some compilers automatically detect opportunities to use SIMD
  - Worthwhile double checking if desired output is achieved with `objdump` or `godbolt.org`)
  - *Advantage*: platform independent

- **Compiler attributes**
  - Use `__attribute__((vector_size (…)))` annotations to state your intentions
  - *Advantage*: platform independent
  - Compiler will generate non-SIMD code if the platform does not support it

- **Compiler intrinsics**
  - Explicit, instruction-level control.

Auto-vectorization example

```c
#include <stdlib.h>
#include <stdio.h>
int main (int argc, char **argv){
    int a[256], b[256], c[256];
    for(unsigned int i = 0; i < 256;
i++){
        a[i] = i + 1;
        b[i] = 100 * (i + 1);
    }
    for(unsigned int i = 0; i < 256;
i++) {
        c[i] = a[i] + b[i];
    }
    printf("c = [ %i, %i, %i, %i ]\n"
,

c[0], c[1], c[2], c[3]);
    return EXIT_SUCCESS;
}
```

- Using x86-64 gcc 16.2 (flag –O3)
  - `movdqa` – move aligned packed integer value
  - `paddd` – parallel add packed integers (d stands for 32-bit values)
  - `movaps` – move aligned packed single precision FP value

```
.L3:
    movdqa  (%rdx,%rax), %xmm0
    paddd   (%rcx,%rax), %xmm0
    movaps  %xmm0, 2048(%rsp,%rax)
    addq    $16, %rax
    cmpq    $1024, %rax
    jne     .L3
```

Register	Use
%rax	byte offset
%rdx	a
%rcx	b
%xmm0	temporary

Increment i by SIMD length of 16 bytes, and check the loop condition.

Compiler attribute

```c
#include <stdlib.h>
#include <stdio.h>
typedef int v4si __attribute__((vector_s
ize (16)));
union int_vec {int   val[4];v4si  vec;};

typedef union int_vec int_vec;
int main (int argc, char **argv){
    int_vec a, b, c;
    a.val[0] = 1;   a.val[1] = 2;
    a.val[2] = 3;   a.val[3] = 4;
    b.val[0] = 100; b.val[1] = 200;
    b.val[2] = 300; b.val[3] = 400;
    c.vec = a.vec + b.vec;
    printf ("c = [ %i, %i, %i, %i ]\n",

c.val[0], c.val[1], c.val[2], c.val[3]);
    return EXIT_SUCCESS;
}
```

- Using x86-64 gcc 16.2 (flag –O0)
  - Data transfers scalar ↔ SIMD is done through memory

```
    movl    $1, -16(%rbp)
    movl    $2, -12(%rbp)
    movl    $3, -8(%rbp)
    movl    $4, -4(%rbp)
    movl    $100, -32(%rbp)
    movl    $200, -28(%rbp)
    movl    $300, -24(%rbp)
    movl    $400, -20(%rbp)
    movdqa  -16(%rbp), %xmm1
    movdqa  -32(%rbp), %xmm0
    paddd   %xmm1, %xmm0
    movaps  %xmm0, -48(%rbp)
```

Register	Use
%xmm0	b, c
%xmm1	a

Coding for SIMD - intrinsics

- Use the SIMD registers explicitly using **intrinsics,** without having to write assembly code

- **Advantages:**
  - As a programmer, you have good control over the instructions that are generated
  - The compiler will manage the register allocation (better than hand-written assembly)

- **Disadvantages:**
  - Code no longer portable to different architectures or you need to provide alternative code for non-SIMD processing
  - If not done carefully, automatic glue code (e.g., cast, etc.) may make the code inefficient.
  - Code readability decreases

- **Rule of thumb:**
  - Try auto-vectorization, check the output.
  - Use intrinsics only in the few hot loops where it pays off

Compiler intrinsics

```
#include <stdlib.h>
#include <stdio.h>
#include <immintrin.h>
int main (int argc, char **argv){
    int a[4], b[4], c[4];
    __m128i x, y;
    a[0] = 1;   a[1] = 2;   a[2] = 3;   a[
3] = 4;
    b[0] = 100; b[1] = 200; b[2] = 300; b[
3] = 400;
    x = _mm_loadu_si128 ((__m128i *) a);
    y = _mm_loadu_si128 ((__m128i *) b);
    x = _mm_add_epi32 (x, y);
    _mm_storeu_si128 ((__m128i *) c, x);
    printf ("c = [ %i, %i, %i, %i ]\n",
    c[0], c[1], c[2], c[3]);

    return EXIT_SUCCESS;
}
```

- **Intrinsics:** compilers wrap up asm instructions as functions
  - Can use them by calling a function with the right parameters
  - `_<vector_size>_<intrin_op>_<suffix>`
  - `<vector size>`: `mm` for 128-bit, `mm256` and `mm512`
  - `<intrin_op>`: `add`, `sub`, `mul`, `etc.`
  - `<suffix>`: `ps` (float), `pd` (double), `epi32` (signed int), `epu16` (unsigned 16-bit integer), etc.

- **Example** with AVX 128-bit intrinsics
  - `__m128i` ← `XMM` register for all integer types
  - `_mm_loadu_si128` ← load (unaligned) 128-bits from memory
    - corresponds to `vmovdqu` assembler instruction
  - `_mm_add_epi32` ← vector add four signed 32-bit integers
    - corresponds to `vpaddd` assembler instruction
  - `_mm_storeu_si128` ← store (unaligned) 128-bits to memory

- **Tip:** use Intel intrinsics guide; search by operation → get the instruction, latency and throughput.

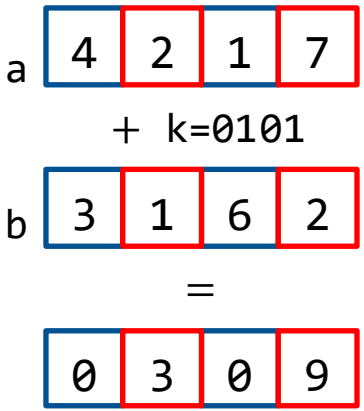
SIMD for data processing

Working with *masks*

- *Conditional* execution of vector elements
- Starting from AVX-512 almost all operations support masking [link]

Zero Masking (selectively ignore some of the SIMD lanes)

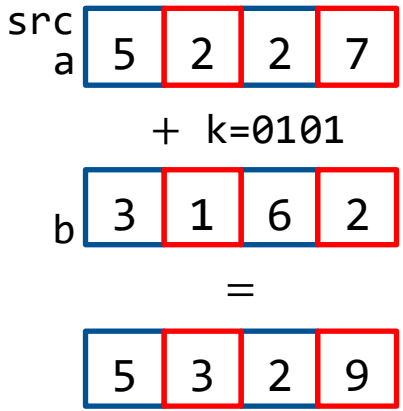
- Example: add elements, but set those not selected by mask to zero:
  - vector add_vector_mask (mask k, vector a, vector b)
  - `__m512i _mm512_maskz_add_epi32 (__mask16 k, __m512i a, __m512i b)`



Working with *masks*

Masking with Merging / Blending

- Blend only instruction
  - vector blend_vector_mask(mask k, vector a, vector b)
  - `__m512i _mm512_mask_blend_epi32 (__mmask16 k, __m512i a, __m512i b)`
- Blend new result with previous result (“merge”)
  - vector add_vector_mask (vector src, mask k, vector a, vector b)
  - `__m512i _mm512_mask_add_epi32 (__m512i src, __mmask16 k, __m512i a, __m512i b)`



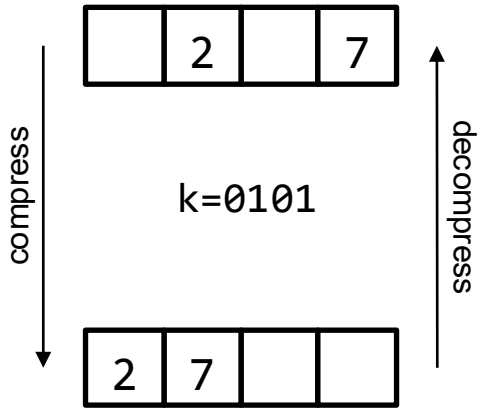
Note the difference between `mask` and `maskz` from previous slide.

Working with *masks*

Compress and expand

- Compress: `__m512i _mm512_maskz_compress_epi32(__mmask16 k, __m512i a)`
  - Also to memory: `compressstoreu`

- Expand: `__m512i _mm512_maskz_expand_epi32(__mmask16 k, __m512i a)`
  - Also to memory: `expandloadu`



SIMDifying our Toy Sum Example

Which part of the code can be accelerated?

- Solely relying on –O3 to let the compiler auto-vectorize your code may not work

  - *e.g.*, like in our toy sum example
    - Complex control flow (branches, breaks), data-dependencies, not-countable loop, no predication

      ```
      g++ -O3 -march=native -fopt-info-vec-missed -fopt-info-vec-optimized file.cpp
      ```

      - `file.cpp:60:6: missed: not vectorized: control flow in loop.`
      - `file.cpp:62:9: missed: value of iteration count not known.`
      - `file.cpp:64:11: missed: loop contains multiple exits.`

- But we know that scanning for the ‘newline’ character ‘\n’ can be quite time consuming and we can speed it up
  - Character parsing is not a recognized SIMD idiom for our compilers…

SIMD-Within-A-Register (SWAR)

- Before we SIMD-ify with vector instructions, let's work with bit-level parallelism within a register
- SWAR – uses one machine word to perform multiple identical operations at once, by careful masking and shifting → just like the SIMD we saw before.

```
[ byte7 | byte6 | byte5 | byte4 | byte3 | byte2 | byte1 | byte0 ]
```

- Warm up:
  - **Q1: How can I fill a 64-bit register with eight copies of the same byte b?**
    - A: `uint64_t pattern = b * 0x0101010101010101ull;`
  - **Q2: How do we find zero bytes**? In each byte of z: 0x80 if that byte of v was 0x00, else 0x00.
    - A: `uint64_t v = ...;` *// 8 bytes*
    - `uint64_t z = (v - 0x0101010101010101ull) & ~v & 0x8080808080808080ull;`

Q2: finding the zero bytes

```
v = [ b7 | b6 | b5 | b4 | b3 | b2 | b1 | b0 ]  ← 8 bytes inside one 64-bit word
z = (v - 0x0101010101010101ull) & ~v & 0x8080808080808080ull;
```

Case i) bi=0x00

```
bi          = 00000000
bi - 1      = 11111111   (mod 256)
~bi         = 11111111
A & B       = 11111111   (A = bi-1, B = ~bi)
(A&B) & 80  = 10000000   → 0x80
```

Case ii) bi = 0x41 (‘A’)

```
bi          = 01000001
bi - 1      = 01000000
~bi         = 10111110
A & B       = 00000000
(A&B)&80    = 00000000   → 0x00  (no flag)
```

```
v             = [ 41  00  7F  80  00  10  FF  01 ]
A = v - 01…01 = [ 3F  FF  7E  7E  FF  0F  FE  00 ]
B = ~v        = [ BE  FF  80  7F  FF  EF  00  FE ]
A & B         = [ 00  FF  00  7E  FF  0F  00  00 ]
(A&B) & 80    = [ 00  80  00  00  80  00  00  00 ]  ← z
```

Computing the sum with SWAR

- Now replace the ‘0’ with ‘\n’ and write the C++ code.

```
uint64_t block=*reinterpret_cast<const uint64_t*>(iter);
constexpr uint64_t high = 0x8080808080808080ull;
constexpr uint64_t low = 0x7F7F7F7F7F7F7F7Full;
constexpr uint64_t pattern = 0x0A0A0A0A0A0A0A0Aull;
uint64_t ascii_mask = (~block)&high;
uint64_t matches0A = ~((((block & low) ^ pattern) + low) & high);
uint64_t matches = matches0A & ascii_mask;
```

- The final matches word holds `0x80` in each byte that matched ‘`\n`’
- The pattern shown here generalizes: you could change ‘`\n`’ to any byte value (e.g., ‘`|`’) to find delimiters 8 at a time.

Computing the sum with SWAR

- We can speed up searching for `\n` by using block operations → SWAR (SIMD within a register).

```
uint64_t block=*reinterpret_cast<const uint64_t*>(iter);
constexpr uint64_t high = 0x8080808080808080ull;
constexpr uint64_t low = 0x7F7F7F7F7F7F7F7Full;
constexpr uint64_t pattern = 0x0A0A0A0A0A0A0A0Aull;
uint64_t ascii_mask = (~block)&high;
uint64_t matches0A = ~((((block & low) ^ pattern) + low) & high);
uint64_t matches = matches0A & ascii_mask;
```

- First execution: 0.65 seconds
- Second execution: 0.38 seconds

Computing the sum with SSE2/AVX

- Let's now do it with actual SIMD instructions (e.g., AVX), trying to find character c

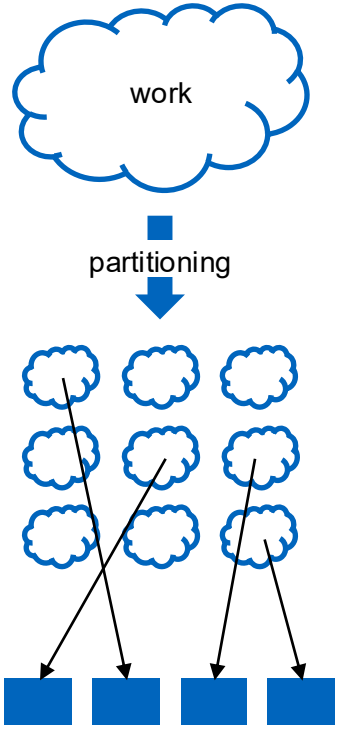
```
auto block = _mm256_loadu_si256(reinterpret_cast<const __m256i*>(iter));
uint64_t matches = _mm256_movemask_epi8(_mm256_cmpeq_epi8(block, _mm256_set1_epi8(c)));
```

- Like SWAR, AVX2 here processes several bytes at once, but using hardware SIMD lanes instead of clever scalar arithmetics
- You can also explore the compiled assembly code at home with godbolt
- Performance
  - First run: 0.7s
  - Second run: 0.42s

Task-level parallelism
(multi-threading, parallel computing)

Basic concepts

- **Work partitioning** (expressing parallelism)
  - **Work** must be split in **parallel tasks**
  - Also known as domain decomposition
- **Task granularity**
  - How much work a task performs?
  - Too little → large overhead
  - Too much → difficult for efficient load balancing
- **Scheduling**
  - Tasks must be mapped into execution contexts
- **Correctness**
  - Order of reads and writes is non-deterministic
  - **Synchronization** is required to enforce the order



Work Partitioning

- Problem: ***how to split the job?***

- Divide the overall problem into independent sub-problems that threads can execute concurrently.

- *Example*: we want to sum the elements of an array
  - Domain decomposition: partition the array, so each thread can get a chunk of indexes of that array
  - Task: each thread computes a local sum
  - Merge: combine the sum in the end.

- **Q1: What if N is not divisible by the number of threads?** (→ uneven chunks)
- **Q2: What if some chunks have more zeros than others?** (→ unbalanced load / work)
- **Q3: What if we had to sum a sparse matrix with variable row length?** (→ irregular partitioning)

- → partitioning should equalize the ***work*** to be done, not just the ***data size***.

Task granularity

- Problem: ***How big should each chunk be?***

- Granularity is what we call the size or cost for each task
  - Fine-grained → better load balance, but higher overhead.
  - Coarse-grained → less overhead, but potentially idle time.

```
#pragma omp parallel for schedule(static, CHUNK) reduction(+:sum)
for (int i = 0; i < N; ++i) { sum += A[i] * B[i]; }
```

- Run `CHUNK = 1`, `CHUNK = 1000`, `CHUNK = N/threads`
- Observations
  - For small chunks → more synchronization and scheduling overhead
  - For huge chunks → load imbalance (threads finish at different times).

Scheduling

- Problem: ***The runtime policy for assigning tasks to threads.***

- Some commonly used policies:
  - **Static scheduling** – fixed chunks per thread
  - **Dynamic scheduling** – threads pick the next available chunk at runtime
  - **Guided** or **work stealing** – adaptive load balancing

```
for (int i = 0; i < N; ++i)
  do_work(i);  // where do_work(i) takes variable time

void do_work(int i) {
  busy_wait(i % 1000); // longer for some iterations
}

#pragma omp parallel for schedule(static)
#pragma omp parallel for schedule(dynamic)
```

Measuring performance when using parallelism

Scalability

- An overloaded concept:
  - *e.g.*, how well a system reacts to increased load, e.g., clients in a server

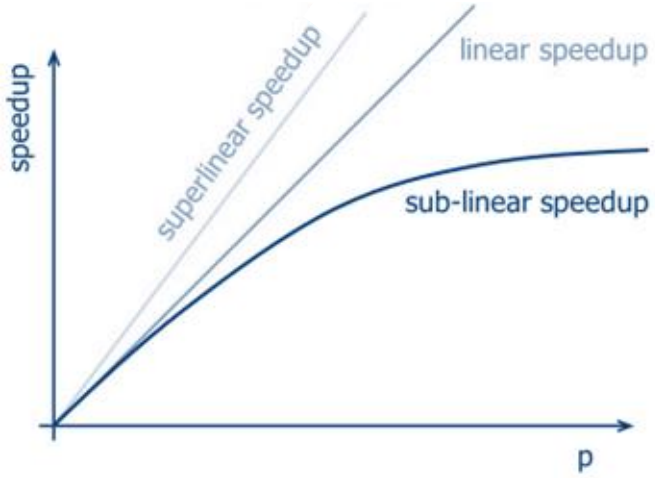
- **Speed-up – how does the RT reduce for the same problem size by adding resources (e.g., cores)?**
  - Speed up for problem size X with N resources: $SpeedUp(N) = RT(1, X)/RT(N, X)$
  - *Ideal:* linear function

- **Scale-up – how does the system deal with larger load (problem size) by adding resources?**
  - Scale up for $N\times$ larger problem by adding $N\times$ resources: $ScaleUp(N) = RT(1, X)/RT(N, NX)$
  - *Ideal:* constant function

- **Scale-out – how does the system deal with larger load (problem size) by adding more machines?**
  - Scale out for $N\times$ larger problem by executing on $N\times$ machines: $ScaleOut(N) = TP(1, X)/TP(N, NX)$
  - *Ideal:* constant function (should behave like Scale-up)

Our focus: speed-up

- Sequential execution time: T_1
- Execution time T_p on p CPUs
- **(parallel) speed-up S_p on p CPUs:** $S_p = \frac{T_1}{T_p}$
  - $S_p = p$: linear speed-up
  - $S_p < p$: sub-linear speed-up / performance loss
  - $S_p > p$: super-linear speed-up / usually poor baseline

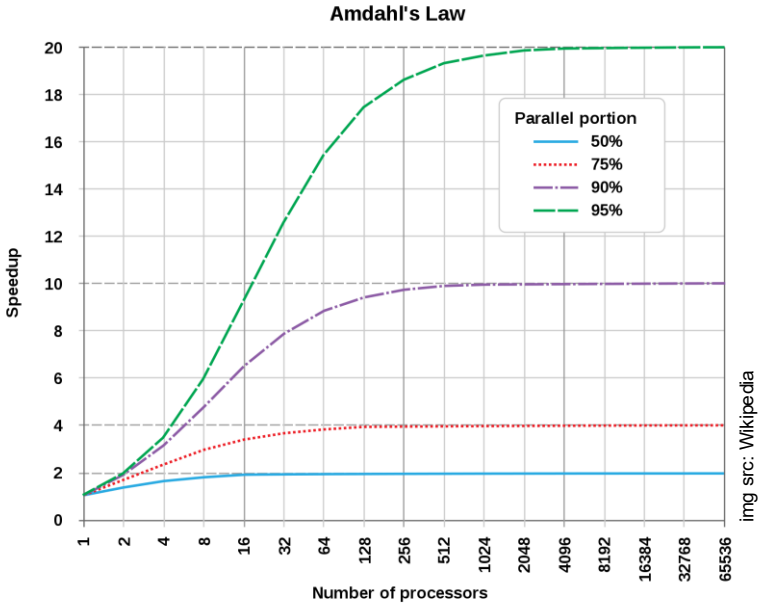


- **Why $S_p < p$?**
  - Programs may not contain enough parallelism
    - Some parts may be inherently sequential
  - Overheads due to parallelization
    - Typically associated with synchronization
  - Architectural limitations
    - Memory contention (memory bound)

Amdahl's Law

Suppose we parallelize an algorithm using n cores and p is the proportion of the task that can be parallelized ($1 - p$ cannot be parallelized)

- The speed up of the algorithm is $\frac{1}{(1-p)+\frac{p}{n}}$
- For infinite parallelism, the speed-up is $\frac{1}{(1-p)}$
- For example, if 90% of the work is parallelized, the maximum speed up is 10
- Ensure that every phase of the algorithm that depends on input data size is parallelized.



Pitfalls in parallel code

- **Non-scalable algorithm**
  - Rethink the algorithm
  - e.g., searching a tree: which one is easier to parallelize BFS or DFS?
- **Load imbalance**
  - Break work into smaller tasks, dynamically schedule these between threads
- **Task overhead**
  - Set a minimum per-thread task size (not too small, not too large)

System constructs for concurrency and parallelism

- **Processes, kernel- and user-level threads and fibers**

- **Process:** an instance of a program that is isolated from other processes on the machine.
  - Has its own private section of the machine's memory.
  - A process abstraction is a virtual computer. Scheduled by the kernel.

- **Thread**: a locus of control inside a running program.
  - A thread abstraction is a virtual processor. Scheduled by the kernel.
  - Threads share all the memory in the process.

- **User-level threads:** act like threads, but implemented in user-space.
  - Can be scheduled preemptively or cooperatively. Invisible to the kernel.

- **Fibers:** light-weight thread of execution that uses co-operative multi-tasking.
  - Fibers yield themselves to run another fiber while executing.

Computing the Sum – parallelism

- **Let's now use multiple cores for processing.**

```
unsigned chunks=thread::hardware_concurrency();
vector<thread> threads;
for (unsigned index=1; index<chunks;++index)
  threads.push_back(thread(sumChunk(index)));
sumChunk(0);
```

Computing the Sum – parallelism

- **Let's now use multiple cores for processing.**

```
unsigned chunks=thread::hardware_concurrency();
vector<thread> threads;
for (unsigned index=1; index<chunks;++index)
  threads.push_back(thread(sumChunk(index)));
sumChunk(0);
```

- We run this on 16 threads
- Performance:
  - First execution: 0.15 seconds
  - Second execution: 0.04 seconds

Compare the Performance

- Warm cache times:

Approach	Time [sec]	Bandwidth [MB/s]
Awk	2.57	282
Python	4.05	179
C++	0.62	1’208
C++ + mmap	0.58	1’250
C++ + swar	0.38	1’907
C++ + parallel	0.04	18’125

- Implementation matters a lot
- Even a desktop box can do multiple GB/s
- Here we saturate everything but DRAM